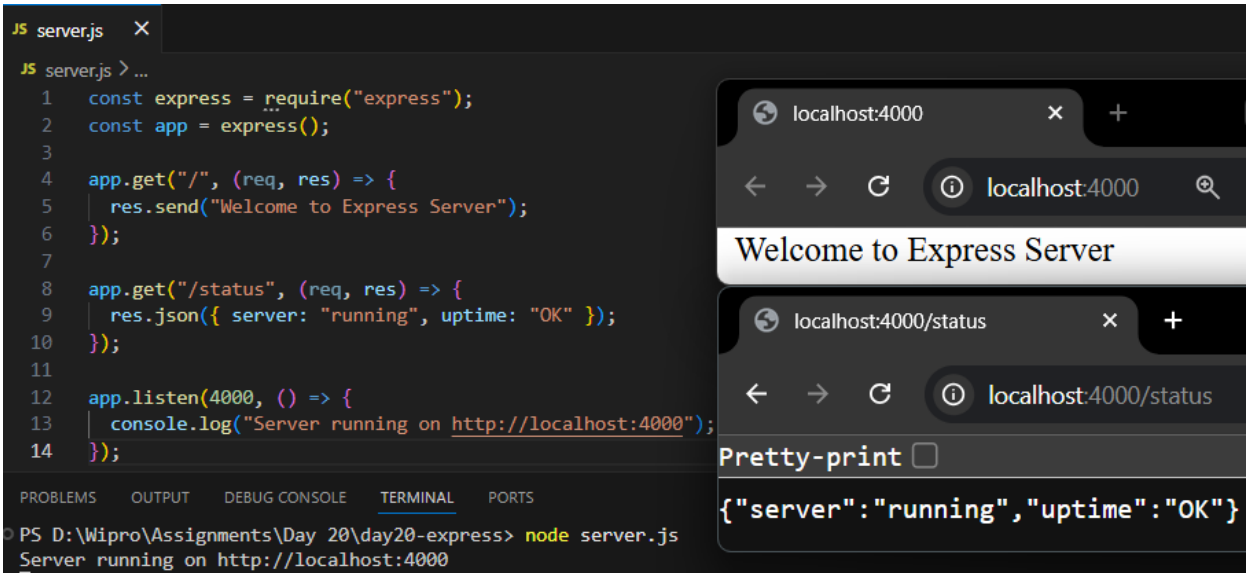
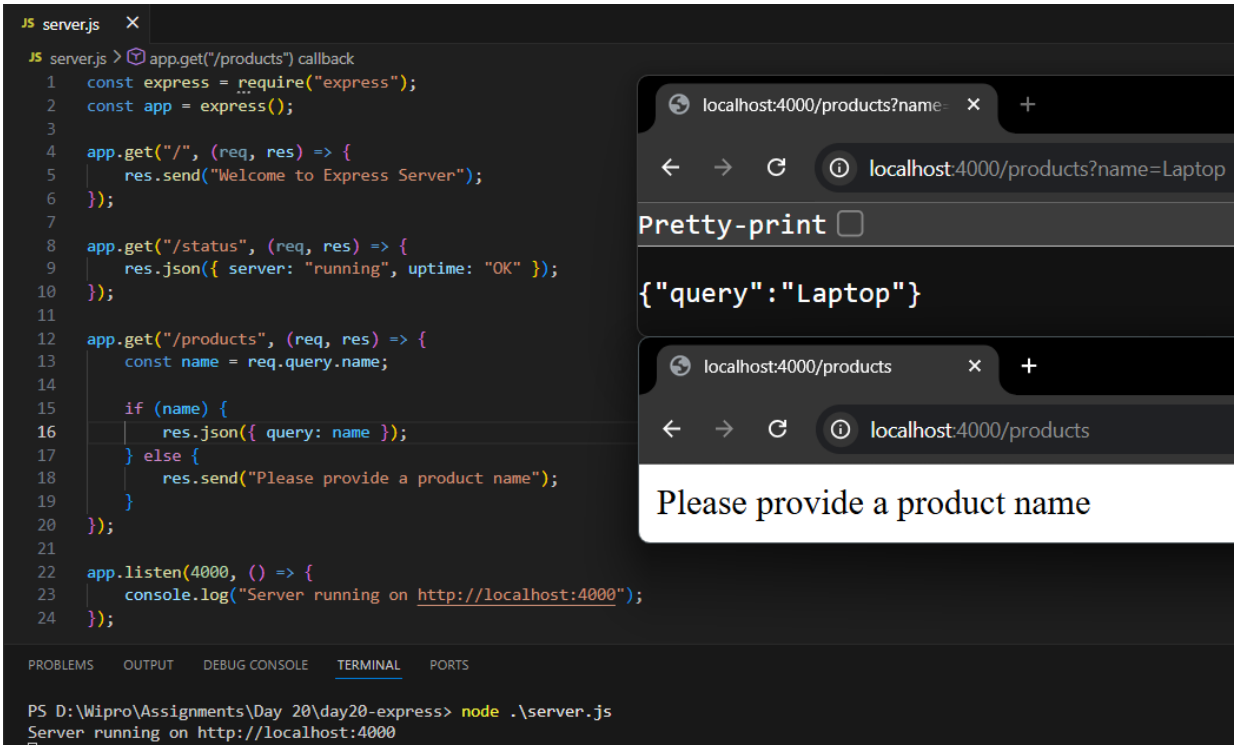
Day 20 Assignment - 1

Description:
Learned how to build a backend server using Express.js with routing, middleware, CRUD operations, and modular code structure.
This day focused on practical API development and maintainable backend architecture.

Challenge 1 – (Setting Up Express Server):
Created a basic Express server and handled a simple GET route with a welcome message.
Also added a status route that returned JSON data.



Challenge 2 (Express Routing & Query Parameters) :
Built a route that used query parameters to search for a product dynamically.
This demonstrated the use of req.query and conditional responses.



Saladi V S S Siva Hemanth Kumar
hemanthsaladi2004@gmail.com
WIPRO NGA - MERN - FY26 - Cohort 3

Challenge 3 (Express Middleware):

Created global middleware to log request details like method, URL, and timestamp.
This helped monitor route access across the application.

```js
const express = require("express");
const app = express();

app.use((req, res, next) => {
  const time = new Date().toLocaleString();
  console.log(`[${time}] [${req.method}] ${req.url}`);
  next();
});

app.get("/", (req, res) => {
  res.send("Welcome to Express Server");
});

app.get("/status", (req, res) => {
  res.json({ server: "running", uptime: "OK" });
});

app.get("/products", (req, res) => {
  const name = req.query.name;

  if (name) {
    res.json({ query: name });
  } else {
    res.send("Please provide a product name");
  }
});

app.listen(4000, () => {
  console.log("Server running on http://localhost:4000");
});
```

```
PS D:\Wipro\Assignments\Day 20\day20-express> node .\server.js
Server running on http://localhost:4000
PS D:\Wipro\Assignments\Day 20\day20-express> node .\server.js
Server running on http://localhost:4000
[3/18/2026, 4:40:29 PM] [GET] /
[3/18/2026, 4:40:44 PM] [GET] /products?name=Laptop
[3/18/2026, 4:40:53 PM] [GET] /products
```

localhost:4000/products?name=Laptop

Pretty-print

{"query":"Laptop"}

localhost:4000/products

Please provide a product name

Challenge 4 (REST API – CRUD Operations , Using POST MAN):

Implemented GET, POST, PUT, and DELETE routes for managing books in memory.
This demonstrated core REST API functionality in Express.

GET http://localhost:4000/books


```
[
    {
        "id": 1,
        "title": "1984",
        "author": "Orwell"
    },
    {
        "id": 2,
        "title": "The Alchemist",
        "author": "Coelho"
    }
]
```


DELETE http://localhost:4000/books/2


```
{
    "title": "Nineteen Eighty-Four"
}
```

```
{
    "message": "Book deleted successfully",
    "deletedBook": [
        {
            "id": 2,
            "title": "The Alchemist",
            "author": "Coelho"
        }
    ]
}
```


PUT http://localhost:4000/books/1


```
{
    "title": "Nineteen Eighty-Four"
}
```

```
{
    "id": 1,
    "title": "Nineteen Eighty-Four",
    "author": "Orwell"
}
```


POST http://localhost:4000/books

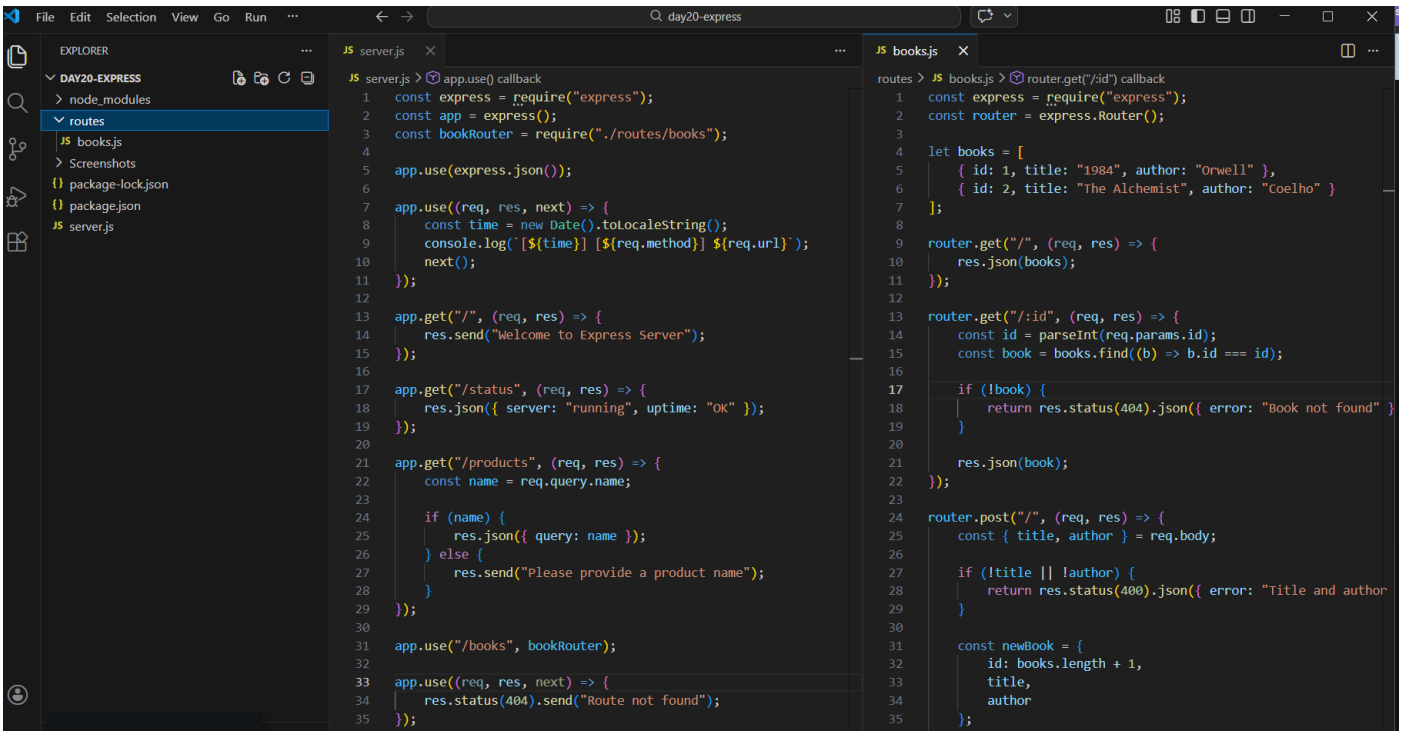

```
{
    "title": "Atomic Habits",
    "author": "James Clear"
}
```

```
{
    "id": 3,
    "title": "Atomic Habits",
    "author": "James Clear"
}
```


Saladi V S S Siva Hemanth Kumar
hemanthsaladi2004@gmail.com
WIPRO NGA - MERN - FY26 - Cohort 3

Challenge 5 (Modular Routing & Error Handling):

Separated book routes into a dedicated router file and added 404 and error-handling middleware. This improved code organization and maintainability.



Saladi V S S Siva Hemanth Kumar
hemanthsaladi2004@gmail.com
WIPRO NGA - MERN - FY26 - Cohort 3